



SWENG 894 Capstone

MicroMatch

Software Architecture and Design Documentation

Chastidy Joanem

M.S. Software Engineering | April 2026

[Github Repository Link](#)

Table of Contents

Table of Contents	1
1. System Overview	2
1.1 Stakeholders and Documentation Use	3
1.2 System Context Diagram	3
2. Architecture Background — Motivation and Design Decisions	4
2.1 Core Motivations	5
2.2 Key Design Decisions	5
3. Architectural Specification	6
3.1 Business Goals and Engineering Objectives	7
3.2 Use Cases and Functional Responsibilities	7
3.3 Non-Functional Requirements (NFRs) and Quality Attributes	8
3.3.1 Quality Attribute Scenarios	8
3.4 Architectural Constraints	9
3.5 Architectural Drivers	10
4. High-Level Architecture	10
4.1 Layered Architecture Diagram	11
4.2 Component Responsibilities	11
5. Low-Level Architecture	12
5.1 Database Schema — Entity-Relationship Model	13
ER Diagram (Simplified)	13
5.2 Design Class Diagram — Domain Model	14
5.3 Backend Module Structure	15
5.4 Frontend Module Structure	16
5.5 Package Diagram – Backend Module Dependencies	17
Mapping Between Architecture Views	19
6. Interaction Diagrams	19
6.1 Authentication Flow (UC-1)	19

6.2 Application Submission and Review Workflow (UC-6, UC-7)	20
6.3 Smart Matching and Recommendation Flow (UC-11, SA-1).....	20
6.4 Project Messaging Flow (UC-12).....	21
7. Architecture Design Decisions and Trade-offs	21
DD-1: Stateless JWT Authentication vs. Session-Based Auth.....	22
DD-2: RBAC via FastAPI Dependency Injection vs. Decorator-based	22
DD-3: Matching Engine as Pure Function vs. SQL Query	23
DD-4: Event-Driven Badge Award vs. On-Read Computation	23
DD-5: Cross-Cutting Logging via Middleware vs. Per-Endpoint	24
DD-6: PostgreSQL for Production / SQLite for Tests	24
7.1 Architectural Trade-off Summary	25
8. NFRs Addressed by the Architecture	25
8.1 Architecture Fitness for Purpose	26

1. System Overview

MicroMatch is a web-based micro-internship marketplace that connects university students with organizations seeking short-term, skill-based project contributors. The platform supports the complete micro-internship lifecycle: user registration, profile creation, project discovery, application submission, deliverable submission and review, project completion, post-project feedback, and platform analytics.

The system is architected as a decoupled, full-stack web application comprising a React Single-Page Application (SPA) frontend, a FastAPI REST API backend, and a PostgreSQL relational database. Authentication is stateless using JWT tokens, and authorization is enforced via Role-Based Access Control (RBAC) across all API endpoints.

1.1 Stakeholders and Documentation Use

Stakeholder	Role	Documentation Use
Students	End User	Understand platform capabilities and workflow
Organizations	End User	Understand project management and applicant access
Administrators	Platform Operator	Monitor system logs and moderate content
Developers	Implementer	Understand module boundaries, design decisions, and data flow
Course Instructor	Evaluator	Assess architecture quality, NFR coverage, and design rationale

1.2 System Context Diagram

The diagram below illustrates MicroMatch in its operational context, showing all three user roles, the system boundary, internal layers, and the web browser client.

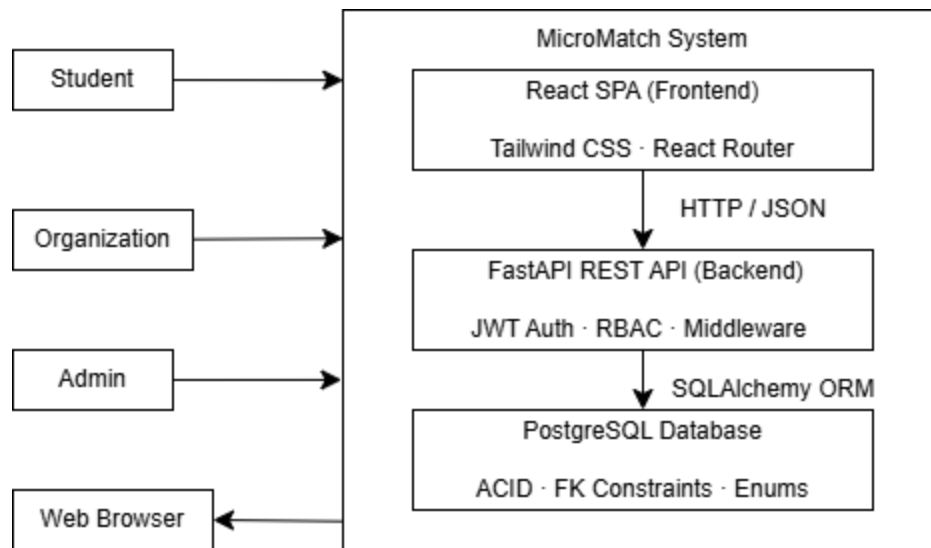


Figure 1 — System Context Diagram

2. Architecture Background — Motivation and Design Decisions

MicroMatch was designed to solve a marketplace coordination problem: universities produce students with theoretical skills but limited practical exposure, while organizations often need short-term project contributors but lack a structured channel to find them. The platform addresses this gap by providing a governed, transactional marketplace with end-to-end workflow support.

2.1 Core Motivations

- **Structured Lifecycle:** The platform must support a complete lifecycle (post → apply → accept → deliver → complete → feedback) rather than just a job board. This motivated a stateful application workflow with clearly defined status transitions.
- **Trust and Credibility:** Both parties must be able to trust the platform's information. This motivated computed badges, ratings, and verified application status rather than self-reported credentials.
- **Security by Default:** Student and organization data is sensitive. Every endpoint must enforce authentication and role separation. This motivated JWT + RBAC as the primary security architecture.
- **Intelligent Matching:** A simple keyword search is insufficient for skill-based matching. This motivated a weighted, multi-factor scoring algorithm (`matching_engine.py`) as a distinct, testable service layer.
- **Observability:** All requests must be logged with user context for debugging and compliance. This motivated cross-cutting logging middleware rather than per-endpoint logging.

2.2 Key Design Decisions

Decision	Choice	Rationale
API Style	REST (FastAPI)	Stateless, cacheable, easily consumed by any frontend. FastAPI provides automatic OpenAPI docs and Pydantic validation with minimal boilerplate.
Authentication	JWT (stateless)	No server-side session storage required. Token carries role claim, enabling RBAC without a DB lookup beyond token validation.
ORM	SQLAlchemy	Database-agnostic query building, relationship management. Keeps SQL logic in Python rather than raw strings.

Decision	Choice	Rationale
Frontend	React + Vite	Component-based SPA enables reusable UI primitives and client-side routing without full-page reloads.
Styling	Tailwind CSS	Utility-first CSS maintains a consistent design token system via tailwind.config.js without a separate component library.
Database	PostgreSQL	ACID compliance, FK constraints, and ENUM support required for the transactional application workflow.
Badge System	Event-driven compute	Badges recomputed on project completion/feedback events. Prevents gaming and ensures accuracy.
Recommendation	Pure function service	matching_engine.py is a stateless pure function: deterministic, testable, and reusable outside the API context.

3. Architectural Specification

3.1 Business Goals and Engineering Objectives

Business Goal	Engineering Objective
Connect students and organizations efficiently	Smart matching algorithm with $O(N \log N)$ ranking and personalized recommendation feed
Support a complete internship lifecycle	Stateful workflow with defined status transitions: pending → accepted/rejected → delivered → completed
Build trust between parties	Verified completion badges, feedback ratings, and portfolio visibility on public student profiles
Protect user data and enforce access control	JWT authentication + dependency-injected RBAC; project-scoped messaging permissions
Enable platform observability	Cross-cutting logging middleware capturing every request with user context and response status
Scale to production	Containerized PostgreSQL via Docker, stateless API, decoupled frontend/backend deployments
Maintain high software quality	≥ 90% backend test coverage; 243 automated tests across unit, integration, and E2E layers

3.2 Use Cases and Functional Responsibilities

UC	Use Case	Actor	FR
UC-1	Register and Login	All	FR-1, FR-2
UC-2	Create / Edit Student Profile	Student	FR-3, FR-19
UC-3	Create / Edit Organization Profile	Organization	FR-4
UC-4	Post and Manage Projects	Organization	FR-5
UC-5	Browse and Search Projects	Student	FR-6
UC-6	Apply to a Project	Student	FR-7
UC-7	Review and Accept/Reject Applications	Organization	FR-8, FR-9
UC-8	Submit and Review Deliverables	Student / Org	FR-10, FR-11
UC-9	Mark Project Complete	Organization	FR-12

UC	Use Case	Actor	FR
UC-10	Submit Feedback	Student / Org	FR-13
UC-11	View Smart Recommendations	Student	SA-1
UC-12	Send and Receive Project Messages	Student / Org	FR-17
UC-13	View Analytics Dashboard	Student / Org	FR-18
UC-14	Receive and Read Notifications	All	FR-16
UC-15	Admin Moderation and Log Review	Admin	FR-14, FR-15

3.3 Non-Functional Requirements (NFRs) and Quality Attributes

ID	Quality Attribute	Requirement	Priority
NFR-1	Security	All endpoints require JWT authentication. Role violations return 403. Project messaging requires verified participant status.	High
NFR-2	Correctness	All business metrics (badges, ratings, analytics) must be computed from live DB records. No stale cached values.	High
NFR-3	Testability	≥ 90% backend coverage. All business logic unit-testable without external services. Pure functions preferred.	High
NFR-4	Performance	Analytics and recommendation queries use SQL aggregates. Single DB round-trip per endpoint.	Medium
NFR-5	Maintainability	Strict separation between layers (routers, schemas, models, utils). Business logic must not leak into schema or model files.	Medium
NFR-6	Observability	Every HTTP request logged with user_id, role, path, method, status code, and timestamp via middleware.	Medium
NFR-7	Deployability	Backend and database containerized via Docker Compose. Frontend deployable independently.	Medium
NFR-8	Usability	Loading states, error states, and empty states present on all async UI components. Mobile-responsive layout.	Low

3.3.1 Quality Attribute Scenarios

Quality attribute scenarios make NFRs concrete by specifying stimulus, source, environment, response, and measurable response measure. The following scenarios correspond to the highest-priority NFRs.

ID	Quality Attribute	Scenario	Response Measure
QAS-1	Security	When an unauthenticated user calls any protected endpoint, the system must reject the request without exposing internal state.	HTTP 401 returned in < 50ms. No data rows returned. No JWT errors logged to the client.
QAS-2	Security (RBAC)	When a student calls GET /analytics/organization (an org-only endpoint), the system must block access and log the attempt.	HTTP 403 returned. SystemLog entry written with user_id and role. No org data returned.
QAS-3	Correctness	When an organization marks a project complete, the student's badges must reflect the new completion count within the same transaction.	award_badges() called before db.commit(). Profile.badges updated atomically with project status change.
QAS-4	Performance	When a student requests recommendations, the system must return ranked results without loading more than the open project set into memory.	Response < 500ms for up to 1,000 open projects. Single SELECT on projects table. No N+1 queries.
QAS-5	Observability	When any HTTP request completes (authenticated or not), a log entry must be persisted with user context.	100% of requests produce a SystemLog row. user_id is null for unauthenticated requests. No request silently skipped.
QAS-6	Testability	All business logic functions must be executable in an isolated test environment without a running PostgreSQL instance.	pytest suite of 243 tests passes against SQLite. Zero tests require Docker or live DB.

3.4 Architectural Constraints

- Language and Runtime: Python 3.11 (backend), Node.js 18+ (frontend build).
- Framework Constraints: FastAPI for the API layer; React for the frontend SPA.
- Database: PostgreSQL for production; SQLite for test isolation. Schema managed via Base.metadata.create_all() (no Alembic in current version).
- Authentication: JWT tokens only. No session cookies, no OAuth2 social login in current scope.
- Deployment: Docker and Docker Compose for local development and database provisioning.
- Frontend-Backend Coupling: The frontend communicates exclusively via the REST API. No direct database access from the frontend.

- Test Database: Tests run against SQLite in-memory database, isolated per test via the `db_session` fixture. Production uses PostgreSQL.

3.5 Architectural Drivers

Architectural drivers are the subset of requirements that have the most significant influence on architecture structure. They are distinct from general requirements in that they force specific design decisions, constrain technology choices, or impose trade-offs that cannot be resolved by simple implementation choices.

The following five concerns are the primary architectural drivers for MicroMatch:

ID	Driver	Type	Influence on Architecture
AD-1	Security and Role Separation	Quality Attribute (Security)	Drives the entire dependency injection strategy for authentication. Every router function must declare its access requirements explicitly. Shapes middleware design and JWT structure.
AD-2	Workflow State Correctness	Functional Requirement	Forces stateful validation logic in every router that touches application lifecycle. Cannot be deferred to the UI layer. Drives the <code>Application.status</code> enum and transition guard patterns.
AD-3	Intelligent and Deterministic Matching	Significant Algorithm (SA-1)	Forces the matching engine to be a pure, side-effect-free function so it can be benchmarked, tuned, and tested independently of the API layer. Drives the separation of scoring logic from router code.
AD-4	Automated Test Coverage $\geq 90\%$	Quality Attribute (Testability)	Drives the dual-database strategy (PostgreSQL prod / SQLite test), the <code>db_session</code> fixture pattern, and the preference for pure functions over stateful classes in business logic.
AD-5	Complete Request Observability	Quality Attribute (Observability)	Drives the cross-cutting middleware approach rather than per-endpoint logging. Forces the <code>SystemLog</code> model to support nullable <code>user_id</code> for unauthenticated requests.

Drivers AD-1, AD-2, and AD-3 are prioritized above AD-4 and AD-5 when trade-offs arise. For example, the choice to use dependency injection for RBAC (AD-1) adds verbosity to endpoint signatures but is non-negotiable. The choice to implement matching as a pure function (AD-3) accepts in-memory scaling limits to preserve testability (AD-4).

4. High-Level Architecture

4.1 Layered Architecture Diagram

MicroMatch follows a strict layered architecture. Each layer has a single responsibility and communicates only with the adjacent layer below it. The cross-cutting `LoggingMiddleware` is the only component that spans all layers.

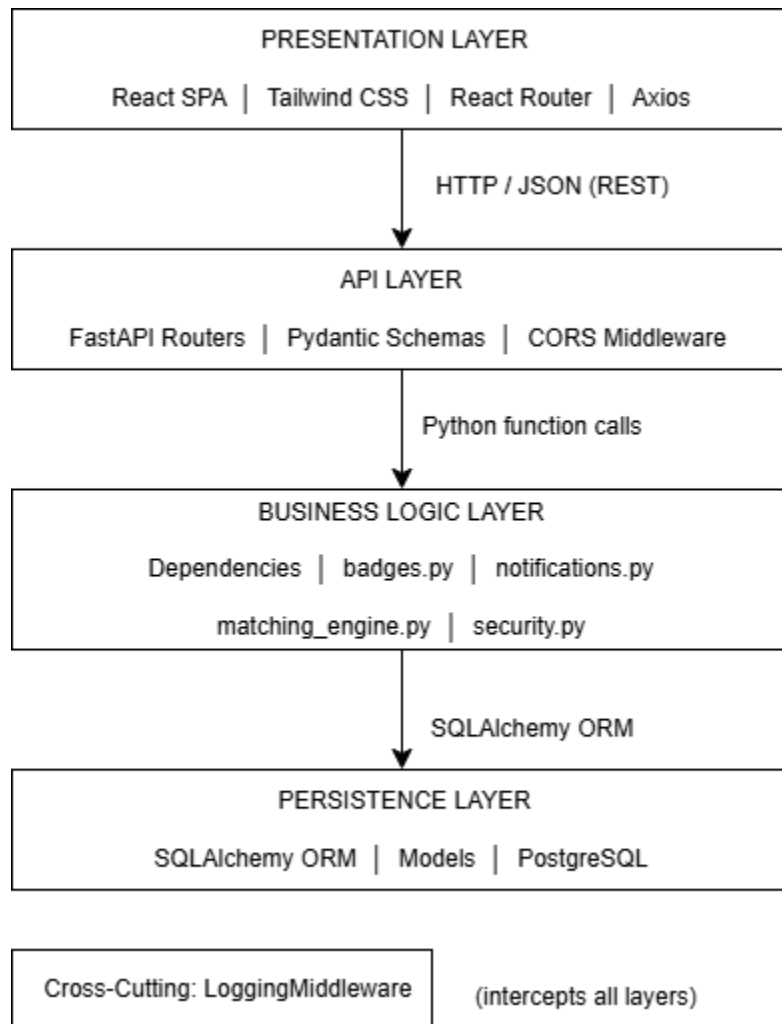


Figure 2 — Layered Architecture Diagram

4.2 Component Responsibilities

Component	Layer	Responsibility
React SPA	Presentation	Renders UI, manages client-side routing, issues Axios API calls, handles auth token storage in memory.

Component	Layer	Responsibility
React SPA	Presentation	Renders UI, manages client-side routing, issues Axios API calls, handles auth token storage in memory.
AuthContext	Presentation	Provides JWT and decoded user identity (role, email, name) to all React components via Context API.
FastAPI Routers	API	Define HTTP endpoints, validate request schemas, delegate to business logic, return response schemas.
Pydantic Schemas	API	Enforce request/response structure. Separate Create/Update/Read/Enhance schemas prevent over-posting.
CORS Middleware	API	Permits cross-origin requests from the React frontend (localhost:3000) to the API (localhost:8000).
LoggingMiddleware	Cross-cutting	Intercepts every request/response, extracts JWT context, writes SystemLog row to the database.
get_current_user	Business Logic	FastAPI dependency. Validates JWT, resolves user_id, loads User from DB. Used by all protected endpoints.
require_role	Business Logic	Dependency factory returning a role checker that raises 403 on role mismatch.
require_project_participant	Business Logic	Dependency enforcing project-scoped access: owning org OR accepted student only.
matching_engine.py	Business Logic	Pure function scoring engine. Computes weighted match scores across 5 factors; returns ranked project list.
badges.py	Business Logic	Event-driven badge computation. Called on project completion and feedback submission. Persists badge string.
notifications.py	Business Logic	Utility to create Notification rows. Does not commit; caller owns transaction for atomicity.
SQLAlchemy Models	Persistence	Define table schema, relationships, and cascade rules. Map Python objects to PostgreSQL rows.
PostgreSQL	Persistence	Authoritative data store. ACID-compliant. Enforces FK constraints, unique constraints, and enum types.

5. Low-Level Architecture

5.1 Database Schema — Entity-Relationship Model

The following table lists all nine database tables with their columns, primary keys (PK), and foreign key relationships (FK). The relationships are shown in the ER diagram below.

Table	Columns (key fields)	Relationships
users	PK id, name, email, hashed_password, role (ENUM), is_active, created_at	Parent to: student_profiles, org_profiles, projects, applications, feedbacks, notifications, system_logs
student_profiles	PK id, FK user_id (unique), university, major, graduation_year, skills, bio, portfolio_links, badges	One-to-one with users (CASCADE DELETE)
organization_profiles	PK id, FK user_id (unique), org_name, industry, website, description	One-to-one with users
projects	PK id, FK organization_id, title, description, required_skills, duration, status, created_at, completed_at	FK → users; one-to-many with applications, feedbacks, project_messages
applications	PK id, FK student_id, FK project_id, status, created_at, updated_at; UNIQUE(student_id, project_id)	FK → users + projects; one-to-one with deliverables
deliverables	PK id, FK application_id (unique), content, status, feedback, created_at, reviewed_at	FK → applications (CASCADE DELETE); one per application
feedback	PK id, FK user_id, FK project_id, rating (1-5), comment, created_at; UNIQUE(user_id, project_id)	FK → users + projects (CASCADE DELETE)
notifications	PK id, FK recipient_id, message, is_read, created_at	FK → users (CASCADE DELETE)
project_messages	PK id, FK project_id, FK sender_id, content, created_at	FK → projects (CASCADE DELETE) + users (CASCADE DELETE)
system_logs	PK id, FK user_id (nullable), role, endpoint, method, status_code, timestamp	FK → users (nullable for unauthenticated requests)

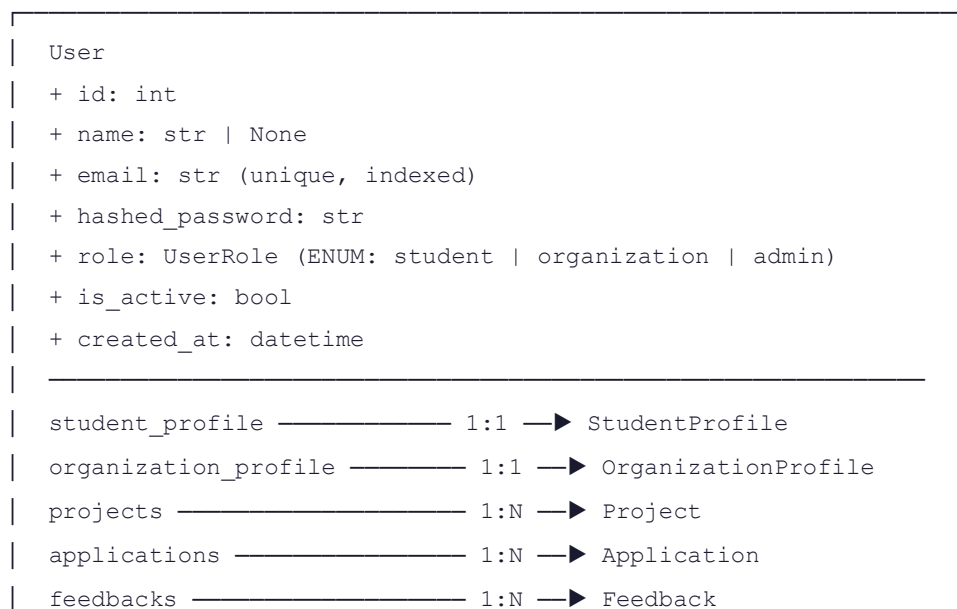
ER Diagram (Simplified)



Figure 3 — Entity-Relationship Model (simplified text diagram)

5.2 Design Class Diagram — Domain Model

The following diagram shows the SQLAlchemy class structure, attributes, and ORM relationships. Each class maps directly to a database table. Relationship navigability is shown with arrow notation.



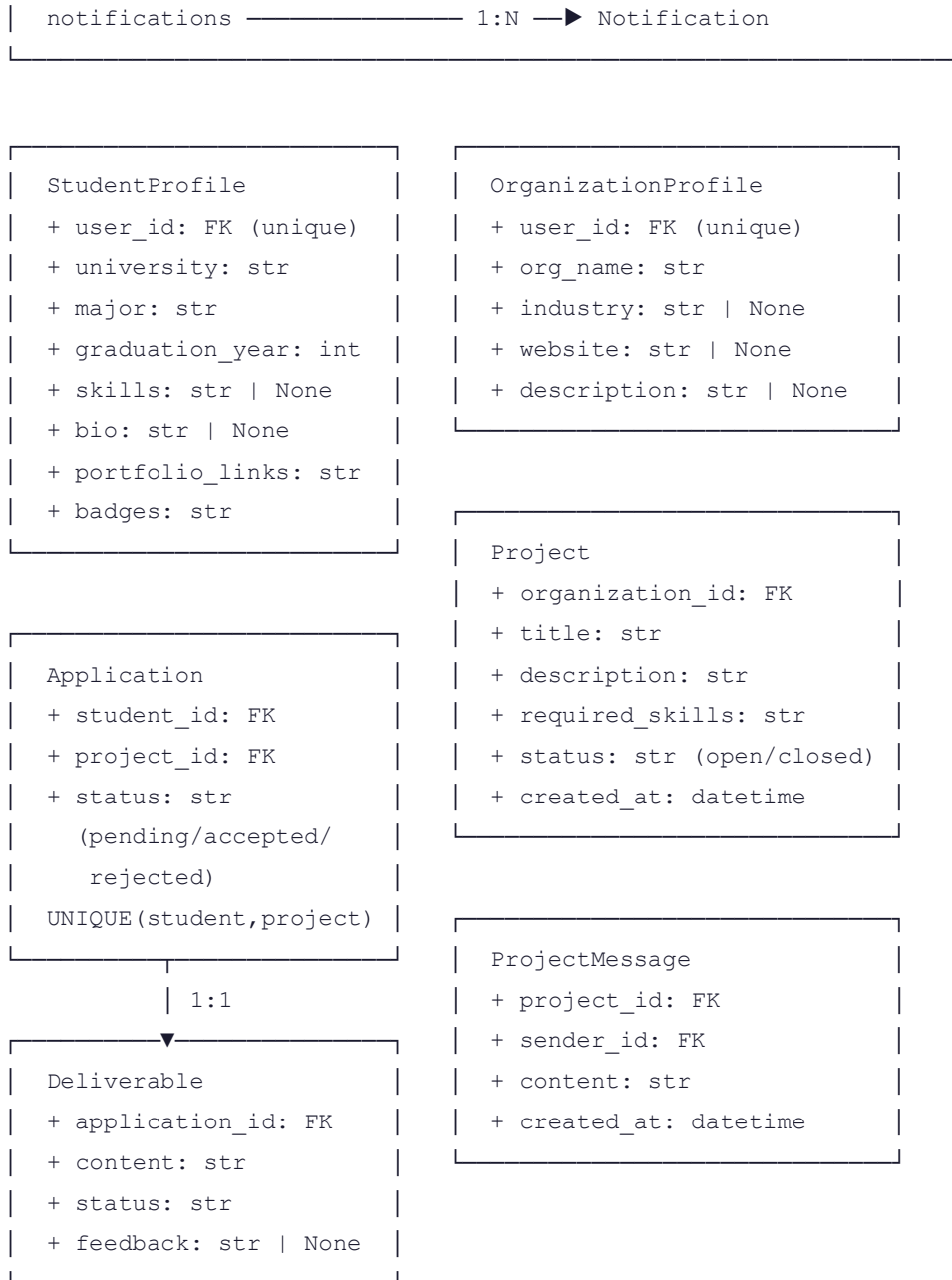


Figure 4 — Design Class Diagram (SQLAlchemy domain model)

5.3 Backend Module Structure

Module	Path	Responsibility
main.py	app/main.py	FastAPI app instantiation, middleware registration, router registration, CORS configuration

Module	Path	Responsibility
models.py	app/models.py	All SQLAlchemy ORM models: User, StudentProfile, OrganizationProfile, Project, Application, Deliverable, Feedback, Notification, ProjectMessage, SystemLog
database.py	app/database.py	Engine creation, SessionLocal factory, get_db dependency for dependency injection
auth.py	app/core/auth.py	JWT creation and decoding: create_access_token(), decode_access_token()
dependencies.py	app/core/dependencies.py	get_current_user, require_role(), require_project_participant() FastAPI dependencies
Routers (×11)	app/routers/	auth, projects, applications, deliverables, feedback, messages, notifications, analytics, recommendations, student_profile, organization_profile
Schemas (×10)	app/schemas/	Pydantic V2 models. Separate Create/Update/Read/Enhance schemas per domain entity to prevent over-posting
matching_engine.py	app/utlis/	Pure function scoring engine. No DB calls. Accepts lists of project dicts and a student profile dict. Five scoring factors.
badges.py	app/utlis/	BADGE_DEFINITIONS list + compute_badges() + award_badges(). Event-triggered via project and feedback routers.
notifications.py	app/utlis/	create_notification() utility. Adds Notification row without committing — caller owns transaction.
security.py	app/utlis/	hash_password() and verify_password() using bcrypt via passlib CryptContext
logging_middleware.py	app/middleware/	BaseHTTPMiddleware subclass. Cross-cutting request/response logger with JWT context extraction.

5.4 Frontend Module Structure

Module	Path	Responsibility
API Clients	src/api/	Per-feature Axios wrapper modules (api.js, messages.js, analytics.js). All HTTP calls centralized. Baseurl and auth header injected via interceptor.

Module	Path	Responsibility
AuthContext	src/context/	React Context providing JWT, decoded user identity (role, email, name), and setProfileComplete globally to all components.
Pages	src/pages/	Route-level components. Each page owns its data fetching, state, and primary user interaction. Examples: Home, StudentProfilePage, MessagesHubPage.
Components	src/components/	Reusable UI primitives: NotificationBell, Layout, ProfileDropdown, RecommendationWidget, AnalyticsDashboard, FeedbackForm, ProfileSetupBanner.
App.jsx	src/App.jsx	React Router route definitions. PrivateRoute wrapper enforces authentication and role-based route access. All routes declared here.
tailwind.config.js	frontend/	Custom design tokens: primary (#fe8997), primaryLight, primaryPale, primaryBg used as utility class extensions across all components.

5.5 Package Diagram – Backend Module Dependencies

The package diagram below shows how the backend modules depend on each other. Arrows represent import dependencies. The diagram enforces the constraint that business logic never imports from routers, and routers never import from each other.



schemas/ (×10) — imported by routers (request/response validation)

Dependency direction rule: routers → core → utils → models

No circular imports. Routers never import from each other.

Figure 5 — Backend Package Diagram (module dependency graph)

Mapping Between Architecture Views

The table below maps each element from the package diagram to its corresponding layer in the high-level architecture (Section 4.1) and its corresponding class in the domain model (Section 5.2).

Package / Module	Architectural Layer	Domain Classes Used	Architectural Driver
routers/	API Layer	All models via SQLAlchemy session	AD-2 (workflow correctness)
core/dependencies.py	Business Logic Layer	User, Application, Project	AD-1 (security/RBAC)
utils/matching_engine.py	Business Logic Layer	None (pure function — no ORM)	AD-3 (intelligent matching)
utils/badges.py	Business Logic Layer	StudentProfile, Application, Project, Feedback	AD-4 (testability)
utils/notifications.py	Business Logic Layer	Notification	AD-1 (security/RBAC events)
middleware/logging_middleware.py	Cross-Cutting (all layers)	SystemLog	AD-5 (observability)
models.py	Persistence Layer	All domain classes	AD-2, AD-4
schemas/	API Layer	N/A (Pydantic, no ORM)	AD-2 (correctness)

6. Interaction Diagrams

The following sequence diagrams illustrate how objects interact to fulfill key use cases. Each step is numbered and maps to the actors, layers, and components defined in Section 4.

6.1 Authentication Flow (UC-1)

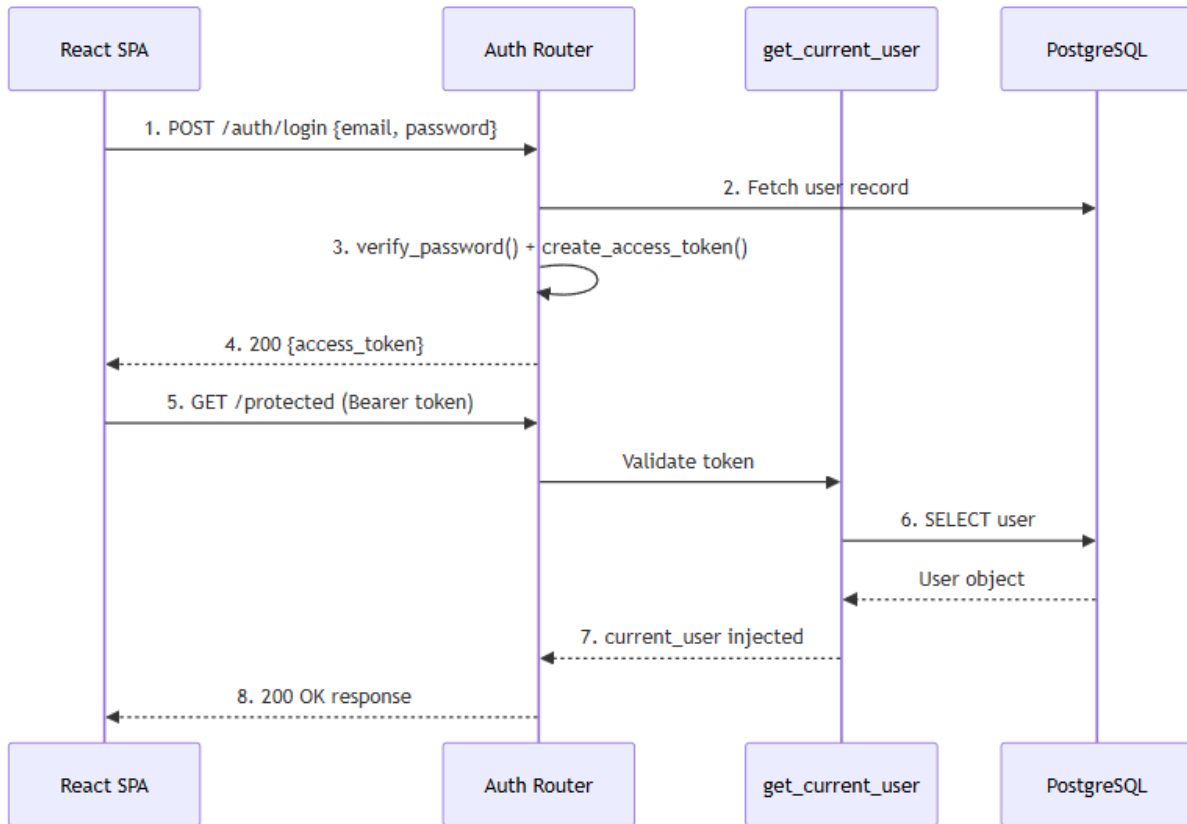


Figure 6 — Authentication and JWT Validation Sequence Diagram

6.2 Application Submission and Review Workflow (UC-6, UC-7)

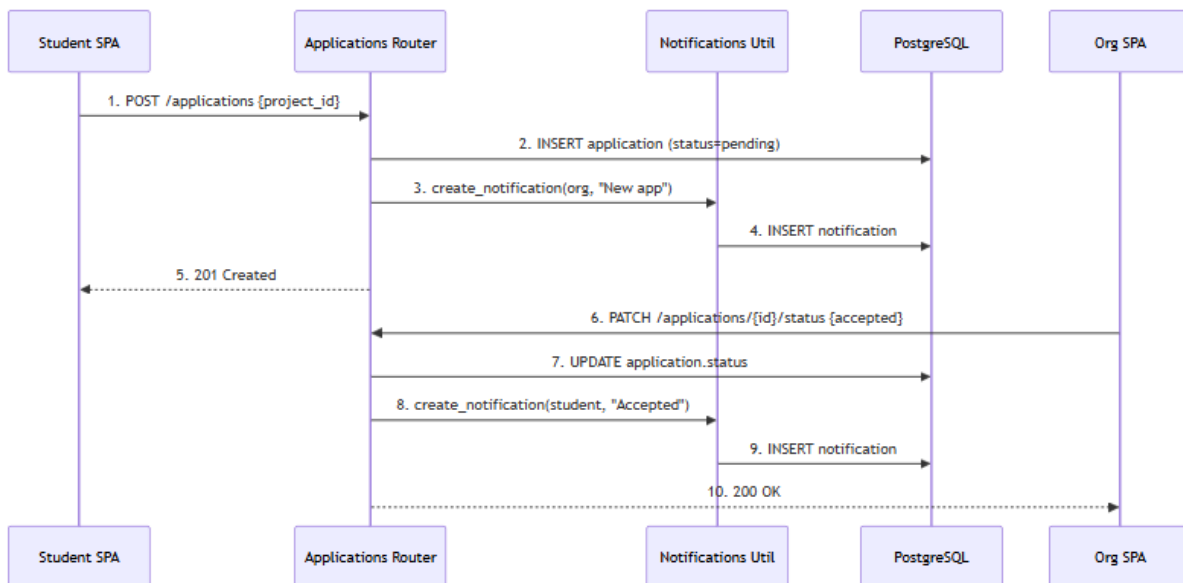


Figure 7 — Application Submission and Review Sequence Diagram

6.3 Smart Matching and Recommendation Flow (UC-11, SA-1)

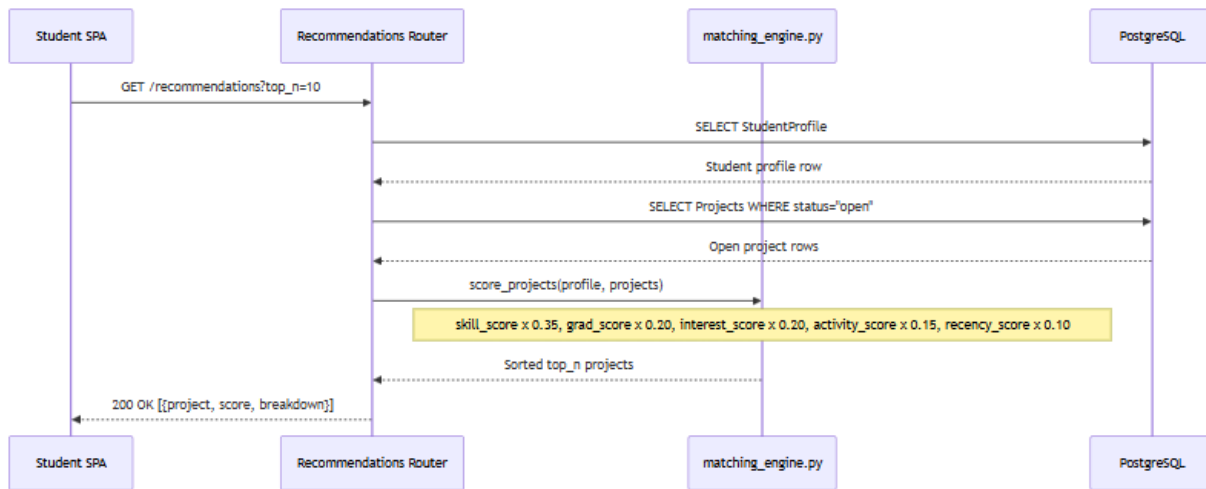


Figure 8 — Smart Matching and Recommendation Sequence Diagram

6.4 Project Messaging Flow (UC-12)

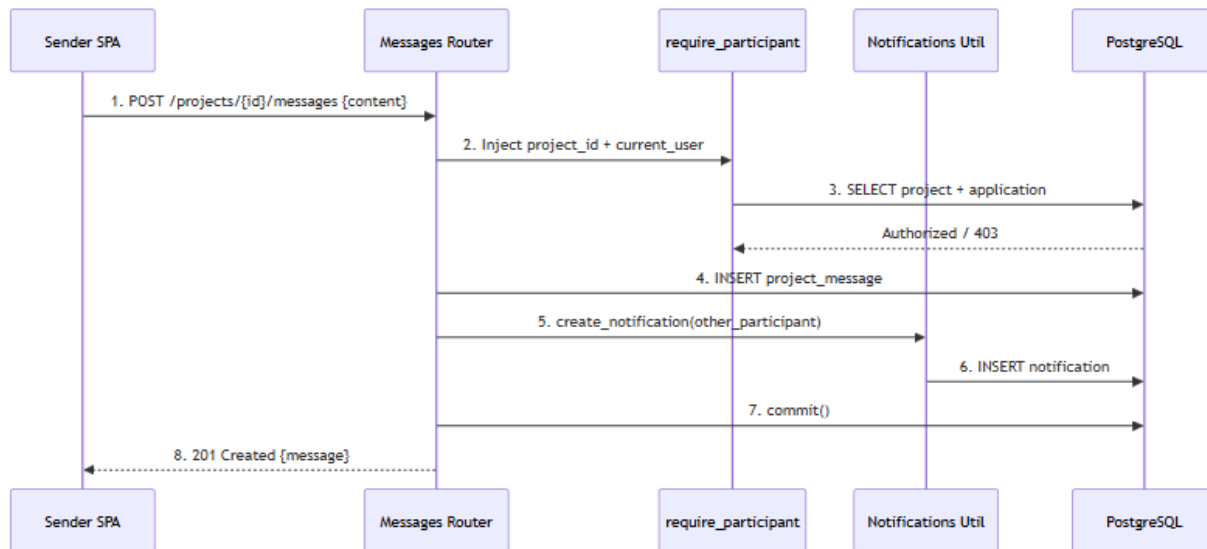


Figure 9 — Project Messaging with Notification Trigger Sequence Diagram

7. Architecture Design Decisions and Trade-offs

This section documents the major architecture design decisions for MicroMatch. Each decision record follows the Architecture Decision Record (ADR) format: context, decision made, rationale, and trade-offs accepted. The decisions are traceable to the architectural drivers defined in Section 3.5.

DD-1: Stateless JWT Authentication vs. Session-Based Auth

Field	Details
Architectural Driver	AD-1
Context	The system must authenticate users across a decoupled frontend and backend. Session-based auth would require shared session storage (e.g., Redis), adding infrastructure complexity.
Decision	JWT (stateless tokens)
Rationale	JWT encodes user identity and role in the token payload, making each API request self-contained. The role claim is verified by <code>require_role()</code> without a DB lookup. The stateless approach aligns with the deployment goal of horizontal scalability.
Trade-offs Accepted	No per-request session DB lookup. Scales horizontally. Role available in every request. Trade-off accepted: token revocation requires a blacklist or short expiry windows; MicroMatch accepts this by using short-lived tokens (expiry configurable via environment variable).

DD-2: RBAC via FastAPI Dependency Injection vs. Decorator-based

Field	Details
Architectural Driver	AD-1, AD-2
Context	Every endpoint must enforce role requirements. Two approaches considered: Python decorators (simpler syntax) and FastAPI's <code>Depends()</code> mechanism (explicit injection).
Decision	Dependency injection using FastAPI <code>Depends()</code>
Rationale	Decorator-based auth couples the auth logic to the function signature and is harder to test without invoking the route. FastAPI's <code>Depends()</code> injects the <code>current_user</code> as a typed parameter, making the dependency explicit, inspectable in OpenAPI docs, and unit-testable without HTTP. The <code>require_role()</code> factory pattern allows a single implementation to serve all roles.

Field	Details
Trade-offs Accepted	Cleaner separation of concerns. Dependencies testable without HTTP client. Visible in automatic OpenAPI documentation. Trade-off accepted: slightly more verbose endpoint signatures compared to a single <code>@authenticated</code> decorator.

DD-3: Matching Engine as Pure Function vs. SQL Query

Field	Details
Architectural Driver	AD-3, AD-4
Context	Recommendation ranking requires multi-factor scoring. Two approaches: embed weights in a SQL CASE expression (inline) or compute scores in Python over pre-fetched data (pure function).
Decision	Pure function service (<code>matching_engine.py</code>)
Rationale	A SQL-based query embeds scoring weights as opaque expressions, making them hard to read, test, or adjust. The pure function approach accepts lists of project dicts and a student profile dict and returns a ranked list. Each of the five scoring factors (skill, graduation, interest, activity, recency) is independently testable. The weight formula is visible in one place.
Trade-offs Accepted	Fully unit-testable without DB. Deterministic and auditable. Weights adjustable without SQL changes. Trade-off accepted: requires loading all open projects into memory. Acceptable at current scale (hundreds of projects); would require server-side pagination above tens of thousands.

DD-4: Event-Driven Badge Award vs. On-Read Computation

Field	Details
Architectural Driver	AD-1, AD-4
Context	Student badges must reflect verified achievements (completed projects, ratings). Two approaches: compute on every profile read (on-demand) or write on triggering events (event-driven).
Decision	Event-driven write to <code>student_profiles.badges</code>
Rationale	On-read computation would add 2+ DB aggregate queries to every profile fetch and cannot be cached without risking stale data. Event-driven writing calls <code>award_badges()</code> on project

Field	Details
	completion and feedback submission, within the same transaction as the triggering event. The profile read becomes a simple column fetch.
Trade-offs Accepted	Profile read is $O(1)$ column fetch. Badge data is transactionally consistent with the triggering event. Trade-off accepted: badge accuracy depends on trigger coverage — adding a new badge type requires identifying all relevant event triggers and running a one-time backfill.

DD-5: Cross-Cutting Logging via Middleware vs. Per-Endpoint

Field	Details
Architectural Driver	AD-5
Context	Every API request must be logged with user context for observability. Two approaches: log in each router function (per-endpoint) or intercept at the middleware layer.
Decision	BaseHTTPMiddleware (cross-cutting)
Rationale	Per-endpoint logging violates DRY, requires discipline across 11 routers, and would be silently skipped if a new endpoint is added without the logging call. BaseHTTPMiddleware intercepts every request and response regardless of which endpoint matches. JWT context is extracted from the Authorization header without requiring the endpoint to pass user information.
Trade-offs Accepted	Zero boilerplate in routers. 100% request coverage by default. Trade-off accepted: middleware must handle JWT parse errors gracefully (<code>user_id = None</code> for invalid or missing tokens) to avoid blocking responses.

DD-6: PostgreSQL for Production / SQLite for Tests

Field	Details
Architectural Driver	AD-4
Context	Tests must be isolated and fast. Running tests against a live PostgreSQL instance requires Docker and adds setup complexity. SQLite in-memory databases are available natively in Python.
Decision	Dual-database strategy

Field	Details
Rationale	The db_session pytest fixture creates a fresh SQLite in-memory database per test, wrapped in a transaction that is rolled back after each test. This provides complete isolation with zero cleanup overhead. SQLAlchemy's ORM abstraction makes the same queries work on both PostgreSQL and SQLite with only minor configuration differences.
Trade-offs Accepted	Fast, isolated tests with no external service dependency. 243 tests run in ~143 seconds locally. Trade-off accepted: PostgreSQL ENUM types are stored as strings in SQLite, requiring careful model design to avoid type mismatch errors.

7.1 Architectural Trade-off Summary

The table below summarizes how each architectural driver maps to the quality attributes it prioritizes and the trade-off accepted. This provides the complete rationale for the most consequential design choices.

Architectural Driver	Prioritized Quality Attributes	Design Decision	Accepted Trade-off
AD-1: Security & RBAC	Security, Correctness	DD-1 (JWT), DD-2 (DI-based RBAC)	Stateless JWT means no real-time token revocation without a blacklist; short expiry windows mitigate this
AD-2: Workflow Correctness	Correctness, Maintainability	DD-2 (router validation)	Strict state transition validation in every router adds per-endpoint code but prevents illegal transitions at the API level
AD-3: Intelligent Matching	Testability, Performance	DD-3 (pure function engine)	Pure function loads all open projects into memory; acceptable at current scale but would require pagination for large data volumes
AD-4: Test Coverage $\geq 90\%$	Testability, Reliability	DD-3, DD-4, DD-6	Dual DB strategy introduces minor SQLAlchemy portability constraints (ENUM as string in SQLite)
AD-5: Observability	Observability, Maintainability	DD-5 (middleware logging)	Middleware must silently absorb JWT parse failures to avoid blocking any request

8. NFRs Addressed by the Architecture

This section maps each Non-Functional Requirement to the specific architectural mechanism that satisfies it, the quality attribute scenario that makes it measurable, and the design decision that motivated the approach.

NFR	Quality Attribute	Architectural Mechanism	QAS	Design Decision
NFR-1	Security	JWT + require_role() dependency injection. require_project_participant() for project-scoped access. 401/403 on violation.	QAS-1, QAS-2	DD-1, DD-2
NFR-2	Correctness	Badges computed via award_badges() within the triggering transaction. Analytics via func.count/avg per request.	QAS-3	DD-4
NFR-3	Testability	Pure function matching engine. DI makes all deps swappable. SQLite test DB per test via db_session fixture.	QAS-6	DD-3, DD-6
NFR-4	Performance	Single-pass SQL aggregates for analytics. In-memory scoring for recommendations. No N+1 queries.	QAS-4	DD-3
NFR-5	Maintainability	Strict module layering. BADGE_DEFINITIONS = one-line change per new badge. Router functions are thin delegates.	—	DD-2, DD-4
NFR-6	Observability	LoggingMiddleware writes SystemLog on every request. Admin endpoint provides filtered retrieval.	QAS-5	DD-5
NFR-7	Deployability	docker-compose.yml provisions PostgreSQL. Stateless API. Frontend builds to static assets independently.	—	DD-1
NFR-8	Usability	All async components have loading skeletons, error states, and empty states. Tailwind responsive grid.	—	UI implementation

8.1 Architecture Fitness for Purpose

The MicroMatch architecture successfully addresses all architectural drivers and satisfies all Non-Functional Requirements within the project constraints. The layered structure ensures clear separation of concerns; the dependency injection strategy makes security enforcement explicit and testable; the pure function matching engine provides a deterministic, auditable recommendation system; and the event-driven badge system maintains data integrity without on-read computation overhead.

The primary remaining architectural gap is the absence of Alembic database migrations. The current Base.metadata.create_all() approach creates tables on first run but cannot alter existing tables, requiring manual ALTER TABLE commands for column additions. This is the highest-priority technical debt item for

any future iteration. A secondary gap is the absence of automated frontend tests (React Testing Library / Vitest), which would complete the full-stack testing strategy.